

DEFENSE PREPARATION GUIDE

Improving Security and Performance of Financial RESTful APIs Using ChatGPT in ASP.NET Core

Zaw Ye Htut Ko | G6519692

Master of Science in Information Technology

Advisor: Assistant Professor Dr. Darun Kesrarat

September 2026

Defense principle: Use the model to propose. Use evidence to decide.

How to Use This Guide

Your target: Deliver the 22 main slides in 27-28 minutes, then use appendix slides only when a professor asks for detailed evidence.

- Do not memorize every sentence. Memorize each slide's key message and first sentence.
- Use Presenter View so the current notes and the next slide remain visible to you.
- Explain charts from left to right: baseline, after result, interpretation, limitation.
- When interrupted, answer the question directly and return with: 'I will connect that answer to the next result.'
- Keep an offline backup of the PPTX, presentation PDF, guide PDF, and evidence screenshots on a USB drive and cloud storage.

Opening Script

Good morning, respected chair and committee members. My name is Zaw Ye Htut Ko, student ID G6519692. Today I will present my thesis on improving a financial ASP.NET Core RESTful API using ChatGPT as a code-improvement advisor. The central idea is that the model proposes changes, but human review and repeatable tool evidence decide whether those changes are accepted.

Closing Script

In conclusion, the study found measurable gains across static quality, security gates, and tail latency, while also identifying limitations in drift measurement and model comparison. The strongest contribution is the controlled audit-to-fix workflow. Thank you for your attention. I welcome your questions.

Presentation Timing Map

Slide	Section	Title	Target
1	Opening	Final Thesis Defense	45 sec
2	Opening	Defense Roadmap	45 sec
3	Problem	One API, Three Failure Modes	75 sec
4	Problem	Research Gap	75 sec
5	Problem	Research Questions	75 sec
6	Method	System Under Study	70 sec
7	Method	Branch-Isolated Research Design	80 sec
8	Method	Four-Phase Evidence Workflow	75 sec
9	Method	Environment and	70 sec

		Seeded Dataset	
10	Method	Evaluation Gates	75 sec
11	Method	Human-in-the-Loop Use of ChatGPT/Codex	75 sec
12	Results	SonarQube: Clean Closure Without Coverage Loss	85 sec
13	Results	OWASP ZAP: Business-Endpoint Gates Cleared	85 sec
14	Results	JMeter: Five Workload Profiles	75 sec
15	Results	JMeter: Large p95 Reductions, One Honest Caveat	100 sec
16	Results	Integrity Was a Non-Negotiable Constraint	75 sec
17	Results	Working Application Evidence	65 sec
18	Comparison	GPT-5.4 vs GPT-5.5: Trade-offs, Not a Universal Winner	100 sec
19	Comparison	New-API v2 Controlled Retest	100 sec
20	Synthesis	Contributions	75 sec
21	Synthesis	Limitations and Future Work	80 sec
22	Closing	Conclusion	55 sec

Planned duration: 27 minutes 35 seconds, leaving time for pauses and brief clarification.

Slide-by-Slide Script

Use these scripts for rehearsal. During the defense, look at the committee and use the notes as prompts rather than reading every word.

Slide 1: Final Thesis Defense

Time: 45 seconds

Key message: This study evaluates AI-assisted remediation through measured software evidence, not trust in generated code.

What to say

Good morning, and thank you for the opportunity to present my final thesis. My research investigates whether ChatGPT, used through the Codex coding environment, can help improve a financial accounting RESTful API in three measurable areas: code quality, security, and performance. The important point is that the model is not treated as an autonomous developer. It acts as an advisor. Every accepted change is reviewed by a human, isolated on a Git branch, and verified with the same tool that identified the original problem. Today I will explain the method, the measured results, the limitations, and the later GPT-5.4 versus GPT-5.5 comparison.

Transition: I will begin with the structure of the defense and the practical problem behind the study.

Do not overclaim: Do not say that ChatGPT independently fixed the system.

Evidence: Report title page | Abstract, p. 3

Slide 2: Defense Roadmap

Time: 45 seconds

Key message: The defense moves from problem to method, evidence, comparison, and defensible conclusion.

What to say

The presentation follows five stages. First, I will define the practical problem and research gap. Second, I will explain the branch-isolated research design and the evaluation tools. Third, I will present the measured SonarQube, OWASP ZAP, and JMeter results. Fourth, I will discuss the GPT-5.4 versus GPT-5.5 comparison and the controlled new-API retest. Finally, I will state the contributions, limitations, and conclusion. Detailed branch, endpoint, and metric evidence is available in the appendix if the committee would like to inspect it during questions.

Transition: The starting point is a practical software-engineering problem in financial APIs.

Do not overclaim: Keep this slide under one minute.

Evidence: Report chapters 1, 3, 4, and 5

Slide 3: One API, Three Failure Modes

Time: 75 seconds

Key message: Security, performance, and maintainability failures require different evidence and must be improved without breaking accounting behavior.

What to say

A financial API can fail in three different ways. First, a security weakness can expose identities, balances, or audit records. Second, a performance bottleneck can make reporting or posting unusable under load. Third, maintainability problems make future defects more likely and slow down remediation. These failures are detected by different tools and normally handled in separate workflows. The research asks whether an LLM can help interpret all three classes of evidence while preserving essential accounting rules, authorization boundaries, audit semantics, and regression tests. This is a harder and more realistic question than asking whether an AI can generate code that compiles.

Transition: Existing research covers code generation extensively, but this tool-guided remediation workflow is less studied.

Do not overclaim: Do not imply that static-analysis findings are equivalent to exploitable vulnerabilities.

Evidence: Sections 1.1-1.4, pp. 9-11

Slide 4: Research Gap

Time: 75 seconds

Key message: Most LLM-for-software-engineering work emphasizes generation; this study evaluates scanner-guided remediation across three independent gates.

What to say

The literature shows strong interest in code generation, code completion, and developer productivity. It also warns that generated code can introduce weaknesses when accepted without review. A smaller area of research is using an LLM as an advisor that receives structured findings from static analysis, security scanning, and load testing, then proposes fixes that are independently remeasured. This thesis addresses that gap with one financial ASP.NET Core API and a combined three-tool workflow. The contribution is therefore not a new AI model. It is an evidence-controlled way to evaluate AI-assisted remediation in a realistic codebase.

Transition: From that gap, the study defines three direct research questions.

Do not overclaim: Describe the contribution as a workflow and empirical case study, not a universal theory.

Evidence: Literature review, pp. 12-27 | Chapter 3 overview, p. 28

Slide 5: Research Questions

Time: 75 seconds

Key message: The study tests measurable change in code quality, security, and performance under a controlled acceptance workflow.

What to say

The research questions align directly with the three evaluation tools. RQ1 asks whether SonarQube findings can be reduced while quality gate, coverage, and duplication do not regress. RQ2 asks whether ZAP High, Medium, and Low alerts on business endpoints can be cleared while authentication and role-based authorization continue to work. RQ3 asks whether JMeter p95 and p99 latency improve across normal, soak, and spike profiles while error rate remains acceptable. This direct mapping is important because the conclusions are based on observable measures, not the model's own explanation of its changes.

Transition: To answer these questions, I used a production-complexity accounting API as the unit of analysis.

Do not overclaim: Do not introduce developer-productivity claims; no human-subject productivity study was performed.

Evidence: Sections 1.5.1-1.5.2, p. 11 | Table 4.1, p. 43

Slide 6: System Under Study

Time: 70 seconds

Key message: The unit of analysis covers complete accounting workflows rather than a toy endpoint.

What to say

The system under study is an ASP.NET Core 8 financial accounting API backed by SQL Server. Its endpoint groups include authentication, users, chart of accounts, journal entries, accounting periods, trial balance, profit and loss, balance sheet, account ledger, and audit logs. This scope matters because the system includes read and write operations, bulk posting, protected administrative actions, financial calculations, and auditability. A React and Vite client was later added as a live validation artifact, but the measured security and performance conclusions come from the API tools, not from the user interface.

Transition: The central control is that each class of remediation is isolated from the same baseline.

Do not overclaim: Call it production-complexity, not a deployed production banking system.

Evidence: API scope, pp. 28-32 | Section 4.6, p. 51

Slide 7: Branch-Isolated Research Design

Time: 80 seconds

Key message: Every tool-specific result is compared against the same baseline on an isolated branch.

What to say

The baseline branch is baseline-v0.1. Static-analysis fixes are applied only on baseline-sonarqube-v1, security fixes only on baseline-zap-v1, and performance fixes only on baseline-jmeter-v1. Each branch is remeasured with the same tool that produced its baseline evidence. The runtime, SQL Server database, host, and seeded data are kept consistent. This design improves attribution because a SonarQube change cannot be credited for a JMeter result from a different branch. It does not create a randomized experiment, but it is a stronger control than applying every suggestion to one changing branch.

Transition: The work on each branch follows the same four-phase evidence cycle.

Do not overclaim: Use 'supports attribution' rather than 'proves causation.'

Evidence: Section 3.1, p. 28 | Table 3.1, p. 29

Slide 8: Four-Phase Evidence Workflow

Time: 75 seconds

Key message: The workflow accepts a fix only after the original measurement is repeated.

What to say

The workflow has four phases. First, capture the baseline and preserve the raw tool finding. Second, provide the finding and relevant source context to ChatGPT or Codex and obtain a candidate fix. Third, a human reviews the recommendation, applies only the acceptable change, and runs focused regression tests. Fourth, the same measurement tool is rerun and the before-and-after evidence is compared. Failed or inapplicable recommendations are not counted as improvements. This scanner-gated loop is the operational contribution of the study and is reusable in a CI/CD review process.

Transition: The environment and seeded data were held constant so those comparisons were repeatable.

Do not overclaim: The diagram describes acceptance control, not full automation.

Evidence: Table 3.1, p. 29 | Prompt and human-effort traceability, pp. 37-38

Slide 9: Environment and Seeded Dataset

Time: 70 seconds

Key message: All comparisons used one fixed local environment and a repeatable accounting dataset.

What to say

All main comparisons used the same Windows 11 host, ASP.NET Core 8 runtime, local SQL Server Financial database, and seeded data process. The dataset contains 120 accounts, 30,000 journal entries, and at least 5,000 posted entries for report-generation paths. The authenticated roles are Admin, FinanceManager, User, and Auditor. JMeter runs in a fixed version 5.5 container, and ZAP uses the stable container image. This environment is repeatable, but it is still one local machine, so absolute latency values should not be generalized to cloud deployment performance.

Transition: Within that environment, five independent gates decide whether a change is acceptable.

Do not overclaim: Do not describe the seeded data as a public financial dataset or real customer data.

Evidence: Table 3.2, p. 29 | Dataset description, pp. 30 and 38-39

Slide 10: Evaluation Gates

Time: 75 seconds

Key message: No single metric is allowed to stand in for overall software quality.

What to say

Five gates are combined. SonarQube measures retained findings, code smells, coverage, duplication, and quality-gate status. OWASP ZAP measures baseline and authenticated alert severity and affected endpoints. JMeter measures average, p95, p99, throughput, error rate, soak behavior, and spike recovery. Dotnet tests check functional regressions, and Git branch and commit evidence preserves traceability. The study does not treat a clean SonarQube result as proof of security, or faster latency as proof of correctness. The tools answer different questions and must be interpreted together.

Transition: The model sits inside this gate structure as an advisor, not as the final decision-maker.

Do not overclaim: Never merge SonarQube, ZAP, and JMeter into one unsupported composite score.

Evidence: Sections 4.1-4.4, pp. 43-48

Slide 11: Human-in-the-Loop Use of ChatGPT/Codex

Time: 75 seconds

Key message: Human review, accounting invariants, and remeasurement bound the model's role.

What to say

The model receives the raw finding, relevant source files, and explicit constraints. It proposes a candidate patch. The human developer checks authorization, validation, idempotency, concurrency, audit writes, accounting periods, balancing, reconciliation, database consistency, and public contracts before accepting the change. Focused tests are run, followed by the original measurement tool. This design reduces the risk of plausible but incorrect AI recommendations. It also means the thesis evaluates an AI-assisted engineering process, not the model in isolation from human judgment.

Transition: With the method established, I will now present the three measured result streams.

Do not overclaim: Avoid saying the model understood accounting rules; the workflow enforced them.

Evidence: Sections 3.1.3-3.1.4, p. 37 | Section 4.5.4, p. 51

Slide 12: SonarQube: Clean Closure Without Coverage Loss

Time: 85 seconds

Key message: Retained findings fell from 28 to 0 while measured coverage and duplication remained stable.

What to say

SonarQube provides the cleanest closure result. Retained C# findings decreased from 28 to zero. Code smells decreased from 21 to zero, while bugs, vulnerabilities, and security hotspots stayed at zero. The final quality gate was OK. Coverage remained 74.3 percent and duplicated-line density remained zero, so the improvement was not achieved by removing tests or duplicating code. Representative changes included replacing a long controller signature with a request DTO, using asynchronous application startup, replacing repeated literals with named constants, and cleaning infrastructure templates.

Transition: Static quality closed cleanly; the security result requires a separate interpretation of gated and informational alerts.

Do not overclaim: Do not call the 28 retained findings 28 vulnerabilities; 21 were code smells.

Evidence: Tables 4.2-4.4, p. 44 | Section 4.1.4, p. 46

Slide 13: OWASP ZAP: Business-Endpoint Gates Cleared

Time: 85 seconds

Key message: All gated High, Medium, and Low business-endpoint alerts were cleared; informational observations remained reportable.

What to say

ZAP was run in passive baseline mode and authenticated API mode. In the baseline scan, Medium alerts decreased from two to zero and Low alerts from six to zero. In the authenticated scan, Low alerts decreased from three to zero. High alerts were zero before and after. Informational observations remained and are reported rather than hidden. The main fixes involved applying security headers before Swagger and OpenAPI middleware, reducing exposed diagnostic UI surface, and updating vulnerable dependencies. The defensible claim is that the pre-registered High, Medium, and Low gate was met on business endpoints, not that every possible security weakness was eliminated.

Transition: Performance was evaluated separately across normal, sustained, and burst workloads.

Do not overclaim: Say 'gated business-endpoint alerts,' not 'the API is completely secure.'

Evidence: Table 4.4, p. 44 | Section 4.1.2, p. 45 | Section 4.5.1, pp. 48-49

Slide 14: JMeter: Five Workload Profiles

Time: 75 seconds

Key message: Normal and stress profiles measure different aspects of system behavior and must not be collapsed into one latency number.

What to say

JMeter uses five profiles. The p50, p100, and p500 profiles evaluate normal through heavy concurrency. The soak profile checks sustained behavior over time, and the spike profile tests response under a sudden burst and the ability to recover. For each profile, the study records p95 and p99 latency, throughput, and error rate. Soak and spike also include drift measures. Tail latency is emphasized because averages can hide slow requests. A lower p95 means that 95 percent of requests complete below that threshold.

Transition: Across these profiles, latency improved substantially, but the stress interpretation remains conservative.

Do not overclaim: Explain p95 correctly; it is not the average response time.

Evidence: Section 4.3, p. 47 | Section 4.5.2, pp. 49-50

Slide 15: JMeter: Large p95 Reductions, One Honest Caveat

Time: 100 seconds

Key message: All five total p95 values improved with zero core-profile errors, while mixed-workload drift remains a limitation.

What to say

The total p95 latency improved in every profile. At p50 it decreased from 49.45 to 13 milliseconds. At p100 it decreased from 261.95 to 34 milliseconds. At p500 it decreased from 176 to 28 milliseconds. Soak decreased from 970.95 to 56 milliseconds, and spike from 6,795 to 1,420 milliseconds. Error rate remained zero for the three core profiles. The main optimizations involved removing report-snapshot writes from read paths, reusing aggregated balances, adding indexes, caching ledger output, and reducing repeated query work. However, the report keeps an honest caveat: the mixed-workload drift heuristic still fails because the final window is compositionally heavier. Endpoint latency improved, but the original drift rule does not fully separate workload composition from degradation.

Transition: Performance improvements are acceptable only if accounting and authorization behavior remains intact.

Do not overclaim: Do not claim that every endpoint or every stability heuristic passed.

Evidence: Table 4.5, pp. 44-45 | Section 4.1.3, p. 45

Slide 16: Integrity Was a Non-Negotiable Constraint

Time: 75 seconds

Key message: Faster or cleaner code was rejected if it weakened authorization, balancing, period, audit, or reconciliation rules.

What to say

This is a financial API, so quality cannot be reduced to scanner counts and latency. The review process preserves JWT authentication and role-based authorization for Admin, FinanceManager, User, and Auditor. It also protects journal balancing, accounting-period constraints, idempotency, concurrency checks, audit writes, database consistency, and reconciliation integrity. The application evidence shows role-aware navigation and read-only behavior for the Auditor role. These controls are not proof of formal financial certification, but they demonstrate that the remediation workflow did not intentionally trade correctness and access control for faster measurements.

Transition: The web application makes several of those API and authorization behaviors visible to a reviewer.

Do not overclaim: Do not claim formal accounting certification or exhaustive security verification.

Evidence: API endpoint and role coverage, pp. 30-32 | Section 4.6, pp. 51-58

Slide 17: Working Application Evidence

Time: 65 seconds

Key message: The screenshots demonstrate the user-visible path through authentication, reporting, audit, and role restrictions.

What to say

The React and Vite client is a validation artifact for the API. It demonstrates login, role-aware navigation, account balances, financial reports, audit logs, and expandable journal-entry debit and credit lines. The Auditor screenshot is particularly useful because it shows restricted navigation and read-only account access. These screenshots support the claim that the API behavior is accessible through a real client. They do not replace the tool measurements or regression tests; they complement them with user-visible evidence.

Transition: The completed report also includes a controlled comparison between GPT-5.4 and GPT-5.5.

Do not overclaim: Do not present screenshots as security-test evidence by themselves.

Evidence: Section 4.6 and Figures 4.1-4.7, pp. 51-58

Slide 18: GPT-5.4 vs GPT-5.5: Trade-offs, Not a Universal Winner

Time: 100 seconds

Key message: SonarQube is a tie, residual ZAP cleanliness favors GPT-5.4, and balanced JMeter behavior with smaller patches favors GPT-5.5.

What to say

The normalized model comparison does not support a universal winner. Both models reduced SonarQube code smells from 21 to zero and reached the same 74.3 percent coverage. GPT-5.4 produced cleaner residual ZAP output. GPT-5.5 passed the business-endpoint gate but retained more Swagger, static-asset, and local-transport observations. In JMeter, GPT-5.4 was slightly faster on selected p100, p500, and soak tail metrics, but it introduced a severe p50 tail-latency regression. GPT-5.5 produced the more balanced profile and the strongest spike p99 and throughput outcome. GPT-5.5 also used fewer source and test files in the ZAP and JMeter patches. The conclusion depends on which quality dimension is weighted most heavily.

Transition: A later controlled retest corrected an important opportunity imbalance in the new-API comparison.

Do not overclaim: Never say GPT-5.5 won every tool or is universally better.

Evidence: Tables 4.7-4.11, pp. 59-60

Slide 19: New-API v2 Controlled Retest

Time: 100 seconds

Key message: After equalizing the correction opportunity, the measured SonarQube and ZAP outcomes are ties; JMeter remains incomplete.

What to say

The new-API v2 retest gave both Sonar candidates the same scan, correction, and rescan opportunity. Both reached zero measured Sonar issues, zero bugs, zero code smells, and an OK gate. GPT-5.4 had a 1.4 percentage-point coverage advantage. Both ZAP candidates reduced normalized business-alert instances from 36 to 12 with identical severity and endpoint coverage. Their full public test suites passed. However, there is no valid GPT-5.4 JMeter remediation candidate; its worktree matches the challenge baseline. Therefore, no overall three-tool winner is recorded. This is not missing decoration; it is a validity boundary that prevents an unfair comparison.

Transition: These results lead to three concrete contributions.

Do not overclaim: Do not score the unavailable JMeter track or treat the baseline as GPT-5.4 output.

Evidence: Table 4.12 and v2 result, pp. 61-62

Slide 20: Contributions

Time: 75 seconds

Key message: The thesis contributes measured evidence, a reusable control workflow, and transparent limits on AI-assisted remediation.

What to say

The thesis makes three contributions. Empirically, it provides measured before-and-after evidence across static quality, security scanning, and load testing on one production-complexity accounting API. Methodologically, it defines a reusable branch-isolated, scanner-gated workflow for accepting or rejecting LLM recommendations. Practically, it preserves traceability from finding to prompt, candidate patch, human decision, focused test, and rerun. The later model comparison adds evidence that smaller patches and balanced behavior can matter as much as winning an individual metric. The thesis does not claim a new algorithm or universal model ranking.

Transition: The value of those contributions depends on keeping their limitations visible.

Do not overclaim: Do not describe a single-system case study as broad statistical proof.

Evidence: Section 5.3, p. 63 | Sections 3.1.3-3.1.4, p. 37

Slide 21: Limitations and Future Work

Time: 80 seconds

Key message: The workflow is reproducible, but external validity, stochastic variance, and incomplete model tracks remain open.

What to say

The study has important limitations. It evaluates one C# API, one database-backed financial domain, and one local host. Repeated stochastic runs were not performed, so model-output variance is unknown. SonarQube, ZAP, and JMeter each have detection limits and cannot replace manual security review or production observability. The mixed-workload drift rule confounds the final request composition with time-based degradation. Finally, the new-API v2 JMeter comparison is incomplete because a valid GPT-5.4 remediation candidate does not exist. Future work should address each of these limitations directly rather than simply expanding the number of metrics.

Transition: Within those boundaries, the final answer to the research problem is clear.

Do not overclaim: State limitations confidently; do not apologize or weaken measured findings.

Evidence: Section 1.6, p. 12 | Section 4.7.6-4.7.7, pp. 61-62 | Section 5.4, p. 63

Slide 22: Conclusion

Time: 55 seconds

Key message: AI-assisted remediation is useful when evidence and human controls decide what ships.

What to say

In conclusion, this study shows that ChatGPT or Codex can be useful as a remediation advisor when it operates inside a controlled engineering process. The original study closed retained Sonar findings, cleared gated business-endpoint ZAP alerts, and achieved large p95 latency reductions. The model comparison shows real trade-offs rather than a universal winner. Human review, focused tests, branch isolation, and independent remeasurement are what make the results defensible. The central lesson is simple: use the model to propose, but use evidence to decide. Thank you. I welcome your questions.

Transition: Open the appendix only when a question requires detailed evidence.

Do not overclaim: End after the final sentence; do not continue summarizing.

Evidence: Chapter 5, p. 63

Professor Question Bank

Answer first, then support the answer. A strong defense answer is normally shorter than one minute unless the professor asks for more detail.

Chair and Contribution

Question 1: What is the main original contribution of this thesis?

Short answer: The contribution is a branch-isolated, scanner-gated workflow that evaluates LLM-assisted remediation across code quality, security, and performance on one production-complexity financial API.

Follow-up answer: The thesis contributes measured before-and-after evidence, preserves finding-to-patch traceability, and shows where conclusions must remain limited. It does not claim a new model or universal ranking.

Supporting slide: S20 and A09

Caution: Do not describe the workflow as fully autonomous.

Question 2: Why is this research important to software engineering?

Short answer: It addresses the slow audit-to-fix cycle by testing whether an LLM can help interpret three different tool outputs while evidence and human review control acceptance.

Follow-up answer: Organizations already use static analyzers, security scanners, and load tests. The study connects those findings to a disciplined remediation loop rather than evaluating code generation in isolation.

Supporting slide: S03-S04

Caution: Do not claim measured developer-productivity improvement.

Question 3: What is the single most important finding?

Short answer: The strongest finding is that measurable gains are possible when the model proposes changes but independent tools and human review decide what is accepted.

Follow-up answer: The Sonar closure, ZAP business gate, and p95 reductions support the process, while the drift caveat and model trade-offs show why uncontrolled trust would be inappropriate.

Supporting slide: S12-S15 and S22

Caution: Choose the workflow finding, not one isolated metric.

Question 4: Why did you include the GPT-5.4 versus GPT-5.5 comparison?

Short answer: It tests whether a newer model changes remediation quality, patch size, and tool outcomes under comparable evidence workflows.

Follow-up answer: The comparison reveals trade-offs: Sonar is tied, GPT-5.4 has cleaner residual ZAP output, and GPT-5.5 is more balanced in JMeter with smaller patches. That is more informative than declaring a winner.

Supporting slide: S18

Caution: Do not claim the comparison is exhaustive across prompts or tasks.

Question 5: What would a practitioner do differently after reading this thesis?

Short answer: They would place LLM advice behind isolated branches, focused regression tests, and the same scanner or load tool that produced the original finding.

Follow-up answer: They should archive prompts and findings, constrain patch scope, preserve domain invariants, and reject suggestions that do not improve the independent evidence.

Supporting slide: S08-S11

Caution: Present this as a recommended workflow, not a guaranteed outcome.

Question 6: Why is this thesis-level work rather than a software project report?

Short answer: It frames research questions, controls branch-level comparisons, defines acceptance criteria, collects multi-tool evidence, analyzes trade-offs, and states validity limits.

Follow-up answer: The web application is an artifact, but the thesis contribution is the empirical design and interpretation across reproducible measurements, not simply implementation of the API.

Supporting slide: S04-S10 and S20-S21

Caution: Do not center the answer on the user interface.

Research Methodology

Question 7: How can you attribute improvements to ChatGPT rather than normal developer work?

Short answer: The thesis evaluates an AI-assisted human workflow, not the model alone; prompts, model suggestions, human decisions, branches, tests, and reruns provide process attribution.

Follow-up answer: A human remained necessary, so the causal unit is the combined process. Branch isolation supports before/after attribution, but it does not separate model skill from reviewer skill completely.

Supporting slide: S07-S11

Caution: Do not claim pure model causality.

Question 8: Why did you use only one codebase?

Short answer: The study prioritizes deep, traceable evaluation of one production-complexity financial API; this strengthens internal traceability but limits external validity.

Follow-up answer: Replication across frameworks, languages, domains, and deployment environments is required before generalizing the effect size.

Supporting slide: S06 and S21

Caution: Acknowledge the limitation directly.

Question 9: Why did you not repeat each model run multiple times?

Short answer: The study controls one archived run per task through branch and prompt traceability, but it does not estimate stochastic output variance.

Follow-up answer: Repeated identical prompts with confidence intervals over fix acceptance, patch size, and final metrics would be the correct next experiment.

Supporting slide: S21 and A08

Caution: Do not suggest deterministic model behavior.

Question 10: Is a before-and-after branch comparison enough to prove causality?

Short answer: No. It supports attribution under controlled environment and branch conditions, but it is not a randomized causal experiment.

Follow-up answer: The design keeps the host, runtime, database, data, and measurement tool stable and isolates fix classes. Remaining confounders include warm-up state, caches, and unobserved branch interactions.

Supporting slide: S07-S09 and A08

Caution: Use 'supports attribution,' not 'proves causation.'

Question 11: How did you prevent cherry-picking successful recommendations?

Short answer: The workflow preserves raw findings, scoped prompts, human-effort traceability, regression tests, and rerun evidence, while the report also records remaining risks and incomplete tracks.

Follow-up answer: The study would be stronger with a complete attempt-level acceptance/rejection dataset across repeated runs. The current report improves transparency but does not eliminate selection bias completely.

Supporting slide: S08-S11 and A09

Caution: Do not claim perfect elimination of selection bias.

Question 12: Why are there different baseline finding counts, 28 retained findings and 21 code smells?

Short answer: Twenty-eight is the retained C# finding set used for remediation traceability; 21 is the SonarQube code-smell metric. They are related but not identical measures.

Follow-up answer: The report keeps both because the retained finding set includes rule-level evidence used by the workflow, while the dashboard metric categorizes code smells separately.

Supporting slide: S12 and A03

Caution: Never label all 28 as code smells or vulnerabilities.

Security

Question 13: Can you claim the API is secure because ZAP has no High, Medium, or Low business alerts?

Short answer: No. I can claim that the pre-registered ZAP business-endpoint gate was met under the tested configuration.

Follow-up answer: ZAP is a dynamic scanner with coverage limits. Manual penetration testing, threat modeling, dependency monitoring, and production controls are still necessary.

Supporting slide: S13 and A04

Caution: Avoid the phrase 'completely secure.'

Question 14: Why did informational alerts remain after remediation?

Short answer: Informational alerts are observations rather than gated vulnerabilities, and some reflect authentication fingerprints, local HTTP transport, Swagger, or static assets.

Follow-up answer: The report retains them for transparency. In the model comparison, these residuals are why GPT-5.4 has cleaner ZAP output than GPT-5.5 even though both pass the business-endpoint gate.

Supporting slide: S13 and S18

Caution: Do not hide or dismiss residual alerts.

Question 15: How did you test authorization?

Short answer: Authenticated scans and role-based tests covered Admin, FinanceManager, User, and Auditor against protected account, period, journal, report, and audit operations.

Follow-up answer: Unauthorized attempts are expected to return 403, and the application evidence also shows restricted Auditor navigation and read-only access.

Supporting slide: S16-S17 and A02

Caution: Do not claim exhaustive authorization proof for every future route.

Question 16: What security fixes had the largest effect?

Short answer: Applying security headers before Swagger/OpenAPI, reducing exposed diagnostic UI, and updating vulnerable dependencies cleared the main gated alert families.

Follow-up answer: The ordering matters because middleware placed after Swagger does not protect those responses. The fix was configuration and dependency focused rather than a claim of redesigning the entire security architecture.

Supporting slide: S13

Caution: Tie each fix to observed alert families.

Question 17: Did you test SQL injection and CSRF?

Short answer: ZAP coverage included injection-related checks, and no gated SQL-injection result remained; CSRF interpretation is limited because the API uses JWT rather than a cookie-based browser session for its core contract.

Follow-up answer: Scanner coverage does not replace manual parameterized-query review. CSRF risk depends on credential transport and browser behavior, so it must be interpreted in the actual authentication design.

Supporting slide: S10, S13, and A04

Caution: Do not claim a manual penetration test was performed.

Question 18: Could caching financial responses expose one user's data to another?

Short answer: Yes if cache keys or authorization boundaries are incorrect, so cache scope, account identity, expiry, and invalidation must be reviewed as security constraints.

Follow-up answer: The thesis treats caching as an optimization subject to authorization and accounting invariants. Future production use should add explicit cross-user cache-isolation tests and distributed-cache threat analysis.

Supporting slide: S15-S16 and A08

Caution: Do not claim the local cache design proves multi-tenant isolation.

Performance and Systems

Question 19: Why focus on p95 and p99 instead of average latency?

Short answer: Tail percentiles show the slow experience hidden by averages and are more sensitive to report-query and contention problems.

Follow-up answer: p95 is the threshold below which 95 percent of requests complete; p99 captures rarer but important slow requests. Throughput and error rate are interpreted alongside them.

Supporting slide: S14-S15

Caution: Do not define p95 as an average.

Question 20: Why did p500 appear faster than p100 in the baseline totals?

Short answer: The profiles have different request mixes and timing, so total p95 values are not guaranteed to increase monotonically with the profile label.

Follow-up answer: This is exactly why endpoint-level results and workload composition matter. The comparison is valid within each profile before versus after, not as a simple cross-profile scalability curve.

Supporting slide: S14-S15 and A05

Caution: Do not invent a hardware explanation without telemetry.

Question 21: What caused the largest latency improvements?

Short answer: The main changes removed report-snapshot writes from read paths, reused aggregated balances, added indexes, reduced repeated queries, and cached account-ledger output.

Follow-up answer: Trial balance, profit and loss, balance sheet, and account ledger endpoints had the largest visible reductions, especially under spike load.

Supporting slide: S15 and Table 4.5 in A05 context

Caution: Do not attribute every millisecond to one change.

Question 22: Why did the original drift heuristic still fail when latency improved?

Short answer: The last measurement window had a heavier request composition, so the aggregate drift metric mixed workload change with time-based degradation.

Follow-up answer: A better design would calculate drift per endpoint or keep request composition constant across windows before aggregating.

Supporting slide: S15 and S21

Caution: Do not describe the drift failure as a hidden system failure or ignore it.

Question 23: Were the load tests run in a production environment?

Short answer: No. They were run on one controlled Windows 11 host with a local SQL Server database and fixed JMeter container.

Follow-up answer: That design improves repeatability for branch comparison but limits absolute performance generalization to cloud, distributed, or multi-node production systems.

Supporting slide: S09 and A08

Caution: Do not call local latency a production SLA result.

Question 24: How do you know caching did not make the accounting data stale?

Short answer: The acceptance workflow preserves posting, balance, audit, and regression checks, but production cache invalidation deserves deeper dedicated testing.

Follow-up answer: The safe rule is to invalidate or version cache entries on relevant journal and account mutations and to test post-write reads explicitly.

Supporting slide: S16

Caution: Do not claim exhaustive cache-consistency proof.

Devil's Advocate

Question 25: Would a competent developer have made the same fixes without ChatGPT?

Short answer: Possibly. The thesis does not compare developer-only productivity; it evaluates whether the AI-assisted workflow produced acceptable measured outcomes.

Follow-up answer: A controlled human-only comparison with time-on-task and review effort would be needed to claim productivity or superiority over expert remediation.

Supporting slide: S20-S21

Caution: Do not claim time savings were measured.

Question 26: Are you simply optimizing for the tools rather than improving the software?

Short answer: That risk exists, so the workflow combines three independent tools, regression tests, accounting invariants, and visible application behavior.

Follow-up answer: Even this triangulation is incomplete. Manual code review and production observability remain necessary, but cross-tool consistency reduces the chance of optimizing one dashboard only.

Supporting slide: S10-S16

Caution: Acknowledge Goodhart's-law risk.

Question 27: Why should we trust model identity and provenance?

Short answer: Identity follows controller-assigned task provenance, isolated branches or worktrees, commit hashes, archived task logs, and paired measurement records.

Follow-up answer: The fresh v2 measurements scan model-produced source code; the tools do not invoke an LLM during measurement. Historical GPT-5.4 process evidence is less complete and is disclosed as such.

Supporting slide: S18-S19 and A01

Caution: Distinguish controller provenance from independent cryptographic attestation.

Question 28: Why did GPT-5.5 have almost as many authenticated ZAP instances as the baseline?

Short answer: Raw instance counts include repeated informational and non-business observations; the gate is interpreted by severity, class, and endpoint, not volume alone.

Follow-up answer: GPT-5.5 cleared the major business security-header problems but retained more Swagger, static-asset, and local-transport residuals, so GPT-5.4 is correctly reported as cleaner on ZAP.

Supporting slide: S18 and A04

Caution: Do not use only the pass label; acknowledge the residual volume.

Question 29: Is the new-API v2 comparison fair if GPT-5.4 lacks a JMeter candidate?

Short answer: It is fair only for the paired SonarQube and ZAP tracks; it is not valid for an overall three-tool ranking.

Follow-up answer: The report explicitly marks JMeter not scored and requires a fresh GPT-5.4 remediation run before any three-tool conclusion.

Supporting slide: S19 and A07

Caution: Never treat the challenge baseline as GPT-5.4 remediation output.

Question 30: What result would falsify your main conclusion?

Short answer: If repeated controlled runs failed to reproduce tool improvements, introduced domain regressions, or required more review effort than conventional remediation, the practical claim would weaken.

Follow-up answer: The thesis conclusion is conditional: usefulness depends on bounded prompts, human review, independent gates, and repeatable evidence. Removing those conditions changes the claim.

Supporting slide: S21-S22

Caution: Answer with falsifiable conditions, not defensiveness.

Rehearsal Plan

Run 1: Content run

Explain each slide without reading and mark unfamiliar terms.

Run 2: Timing run

Finish the main deck in 27-28 minutes and shorten only notes, not evidence.

Run 3: Defense run

Present once, then answer ten hostile questions using the appendix.

Four-Step Question Method

1. Answer the question in one sentence.
2. Give the strongest measured evidence.
3. State the relevant limitation.
4. Stop and invite a follow-up.

Recovery Phrases

- The short answer is...
- The evidence I can support is...
- That is outside the measured scope of this study; the relevant limitation is...
- I do not want to overclaim, so I would separate the measured result from the interpretation.

Three-Day Practice Schedule

When	Practice
Day 1	Read every note aloud. Mark words that feel unnatural and rehearse the opening and conclusion three times.
Day 2	Run the full deck with a timer. Practice the five questions you find most difficult and use appendix slides.
Day 3	Perform one final defense without stopping. Review only mistakes and sleep normally; do not rewrite the presentation.

Final-Day Checklist

Files

- ☐ Editable PowerPoint opens without repair warnings.
- ☐ Presentation PDF matches the final slide order.
- ☐ Preparation guide PDF is available offline.
- ☐ Evidence screenshots and the final thesis PDF are available as an offline backup.
- ☐ A second copy exists on a USB drive or another device.

Room and display

- ☐ Confirm the projector uses 16:9 widescreen mode.
- ☐ Test the HDMI or USB-C adapter before the committee arrives.
- ☐ Disable notifications, chat popups, automatic updates, and screen sleep.
- ☐ Use Presenter View and confirm the notes appear only on your screen.

Speaking

- ☐ Memorize the opening, conclusion, and first sentence of every results slide.
- ☐ Carry water and pause after each major result.
- ☐ Look at the professor who asked the question before looking at the appendix.
- ☐ If uncertain, state the measured boundary instead of guessing.

Last reminder: You do not need to know everything. You need to explain what was measured, what the evidence supports, and where the limitations begin.